

CUDA Optimization of Fused Causal Softmax for Transformer Attention

Work Decomposition, Warp-Level Reduction, Empirical Ablation, and Attention-Level Performance

Arwin Karir

Educational GPU Systems Research Project

Abstract

Context: Causal scaled softmax combines scaling, future-position masking, stable exponentiation, row-wise maximum and denominator reductions, synchronization, and global-memory traffic. This makes it a compact subject for studying how CUDA design choices translate into kernel and application performance. **Objective:** This study evaluates how work decomposition, reduction communication, and launch configuration affect a fused forward-pass FP32 causal scaled-softmax operator, and how much isolated-kernel improvement survives in explicit transformer attention. **Method:** One evolving implementation progressed from one thread per row to one block per row with full shared-memory trees and then compact warp-shuffle reductions. Seven sequence lengths, three launch sizes, three isolated-softmax paths, and three attention paths were measured. Each configuration received 25 warmups and 100 CUDA-event observations in one NVIDIA Tesla T4 session; these observations describe within-session variation, not independent replications. **Results:** All 88 structured softmax cases passed at $\text{rtol} = 10^{-5}$ and $\text{atol} = 10^{-6}$, with maximum absolute error 3.5763×10^{-7} . The warp kernel was $3.22\text{--}8.92\times$ faster than the row-serial implementation, $1.07\text{--}1.91\times$ faster than the shared-tree implementation, and $1.40\text{--}3.94\times$ faster than equivalent PyTorch eager softmax. Explicit attention improved by $1.54\text{--}2.31\times$, but PyTorch scaled-dot-product attention remained faster at every tested shape. **Conclusion:** Cooperative work mapping and compact warp communication materially improved the isolated operator and an explicit composition in the measured session. Unchanged matrix multiplications and attention-wide fusion limited application-level gains, so the evidence does not establish production superiority or cross-device generality.

Keywords: CUDA; GPU optimization; stable softmax; causal attention; transformer attention; kernel fusion; parallel reduction.

1. INTRODUCTION

Transformers organize sequence modeling around attention, which exposes parallel work across tokens while learning data-dependent relationships [1]. Scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right)V, \quad (1)$$

where d is the head dimension and M excludes invalid positions. In causal self-attention, a query at position i may attend only to keys at positions $j \leq i$. PyTorch exposes this operation through scaled-dot-product attention (SDPA) and can dispatch to optimized backends depending on the inputs and runtime [2].

Equation 1 contains two matrix multiplications around a reduction-heavy normalization. Softmax requires a maximum reduction for numerical stability, exponentiation, a denominator reduction, and final normalization. A framework composition also performs scaling and causal masking as separate tensor work. This structure makes causal softmax a compact case study in GPU work decomposition, coalesced memory access, synchronization, shared memory, register-local partials, and warp

communication.

This work studies one primary CUDA source file through Git history rather than maintaining unrelated “naive” and “optimized” implementations. Five research questions (RQs) separate numerical behavior, kernel mechanisms, launch policy, framework comparison, and application translation:

- RQ1** Does the operator preserve the intended causal scaled-softmax contract for the structured FP32 test domain?
- RQ2** How much do cooperative work decomposition and compact warp communication contribute to latency reduction?
- RQ3** Is one threads-per-block choice optimal across the tested sequence lengths?
- RQ4** How does the fused operator compare with equivalent eager and compiled framework softmax paths?
- RQ5** How much isolated-softmax improvement survives in explicit attention, and how does that path compare with production SDPA?

The contributions are: (1) an inspectable fused causal scaled-softmax operator; (2) commit-linked comparisons

of row-serial, shared-tree, and warp-reduction designs; (3) controlled launch and framework comparisons; (4) integration into explicit attention with a production SDPA baseline; and (5) a reproducible evidence archive containing raw samples, environment metadata, figures, profiler exports, and the executed experiment notebook.

The claims are deliberately bounded. All timings were collected in one session on one Tesla T4. Repeated CUDA-event observations reveal conditional timing spread but do not constitute independent machines or sessions. The paper therefore reports descriptive medians, quartiles, ratios, and rank separation, while reserving confirmatory inference for a replicated design.

2. BACKGROUND AND RELATED WORK

2.1. Stable Causal Softmax

For logits $x_1, \dots, x_n$, numerically stable softmax subtracts the row maximum m :

$$p_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}, \quad m = \max_j x_j. \quad (2)$$

Multiplying numerator and denominator by $\exp(-m)$ shows that the probability is mathematically unchanged. Numerically, the largest exponential argument is zero and all others are non-positive, preventing large positive logits from overflowing the exponential. Stable log-sum-exp and softmax evaluation has been studied directly [3], and online normalization work demonstrates how reorganization can reduce memory access [4].

Floating-point addition is not associative, so different parallel reduction orders need not match bit-for-bit. CUDA and PyTorch document the resulting rounding and reproducibility constraints [5]–[8]. Accordingly, this study fixes error tolerances before performance evaluation rather than requiring exact equality or weakening tolerances after observing results.

2.2. CUDA Reductions and Warp Communication

CUDA groups threads into blocks and blocks into a grid. Threads in a block can cooperate through shared memory and block-wide barriers; hardware executes threads in warps, conventionally 32 lanes on the tested architecture [9]. A block reduction can combine one partial per thread through a shared-memory tree. Warp shuffle operations instead exchange register values within a warp, avoiding a shared-memory write–barrier–read sequence at each intra-warp reduction step [10]. A compact two-level design can reduce within each warp, store one result per warp in shared memory, and use the first warp for the final block result.

Parallel reduction is a well-established CUDA optimization subject. Harris develops the canonical sequence of mapping, addressing, and warp-tail transformations, while CUB packages architecture-tuned block collectives [11],

[12]. The hand-written reduction here is intended to expose those engineering choices. It is not evidence that a bespoke kernel would outperform a matched CUB implementation.

Performance does not follow from occupancy or thread count alone. Block size, register use, shared memory, coordination, and row length interact, so launch configuration must be measured rather than inferred [6], [13].

2.3. Fusion and Attention Systems

Kernel fusion can reduce intermediate storage and launch overhead by retaining related work within one kernel. FlashAttention showed that IO-aware tiling can substantially improve exact full attention by reducing high-bandwidth-memory traffic [14]; FlashAttention-2 further improved work partitioning and parallelism [15]. The present operator is deliberately narrower: it fuses the scaled causal softmax stage but leaves QK^T and PV to PyTorch. This boundary makes the difference between a microkernel result and a full-attention result explicit.

Memory-efficient attention can avoid retaining the full quadratic matrix even when arithmetic remains quadratic [16]. Likewise, cuDNN and Triton illustrate two different routes to maintainable optimized kernels: architecture-tuned reusable libraries and tile-level compiler abstractions [17], [18]. These systems motivate stronger future baselines; their absence is treated as a construct-validity limitation.

2.4. Framework Compilation and Empirical Rigor

PyTorch eager execution, graph compilation, and specialized attention backends control different fusion boundaries. PyTorch 2 uses bytecode capture and a compiler stack to optimize graphs while retaining Python flexibility [19]. A comparison against eager execution measures replacement of a multi-operator composition; a comparison against steady-state `torch.compile` asks whether compilation can remove similar overhead; and SDPA evaluates a broader production-oriented boundary.

Systems benchmarking is sensitive to layout, clocks, runtime state, and measurement order. Seemingly innocuous environmental changes can reverse conclusions, motivating raw-data retention, randomized independent sessions, and explicit experimental units [20], [21]. The Roofline model supplies useful memory-versus-compute context [22], but the available profiler fields are insufficient for a quantitative Roofline placement. No bottleneck classification is claimed.

3. METHODOLOGY

3.1. Operation Contract

For $Q, K, V \in \mathbb{R}^{B \times H \times S \times D}$, the score tensor is flattened from $[B, H, S, S]$ to $[R, S]$, where $R = BHS$. For row r ,

the query position is

$$q_r = r \bmod S. \quad (3)$$

Only columns $j \leq q_r$ are allowed. Given a finite positive scale α , the fused operator computes

$$m_r = \max_{j \leq q_r} (\alpha s_{r,j}), \quad (4)$$

$$z_r = \sum_{k \leq q_r} \exp(\alpha s_{r,k} - m_r), \quad (5)$$

$$p_{r,j} = \begin{cases} \exp(\alpha s_{r,j} - m_r) / z_r, & j \leq q_r, \\ 0, & j > q_r. \end{cases} \quad (6)$$

The output preserves shape, dtype, and device. The implementation accepts contiguous two-dimensional FP32 CUDA tensors and implements forward execution only.

The C++ boundary rejects CPU, noncontiguous, non-FP32, non-matrix, or empty inputs; nonpositive or nonfinite scales; and unsupported launch sizes. A CUDA device guard selects the input device, allocation inherits the input options, and launch occurs on PyTorch’s current CUDA stream. Immediate launch errors are reported through the extension boundary. These checks separate the public framework contract from the device algorithm.

3.2. System Architecture

The execution path is shown in Figure 1. Python supplies scores, scale, and launch policy. C++ validates and dispatches; the CUDA kernel stages scaled values, applies the causal predicate, performs maximum and sum reductions, and normalizes. Explicit attention retains the matrix multiplications before and after this boundary.

3.3. Final Kernel Procedure

Algorithm 1 states the final block-per-row procedure without committing to CUDA syntax. Thread t visits $t, t + B, t + 2B, \dots$, where B is the block size. Inactive lanes contribute $-\infty$ to maximum reduction and zero to sum reduction, which preserves the full-warp shuffle mask.

Output reuse removes a separate temporary allocation but imposes three barrier-protected invariants: all staged values must precede consumption of the row maximum; the cross-warp array cannot be reused until the maximum has been read; and all exponentials must precede consumption of the denominator. For $B = 128$, four FP32 warp partials require 16 bytes of dynamic shared memory; $B = 256$ and $B = 512$ require 32 and 64 bytes, respectively. Per row, the method performs three strided passes and $O(S)$ arithmetic with $O(\log 32)$ intra-warp steps plus one compact cross-warp reduction. These counts describe work and storage, not measured latency.

3.4. Hypotheses

Five hypotheses were evaluated after evidence collection:

1. row-serial work scales poorly as row length increases;
2. one block per row improves sufficiently long rows despite coordination;
3. warp reductions improve on full shared-memory trees;
4. the preferred block size varies with sequence length; and
5. isolated-softmax speedup usually exceeds complete-attention speedup.

3.5. Correctness Protocol

The custom output was compared with an independent PyTorch composition at $\text{rtol} = 10^{-5}$ and $\text{atol} = 10^{-6}$. Sequence lengths were 31, 32, 33, 63, 64, 127, 128, 255, 511, 768, and 1023. Eight input families covered normal random values; random values multiplied by 10, 100, and 1000; zeros; equal values; one dominant positive value; and one dominant negative value. Tests checked numerical closeness, exact causal zeros, approximate row sums of one, finiteness, shape, dtype, and device. Complete attention was also compared with explicit PyTorch attention and SDPA.

3.6. Benchmark Protocol

The isolated-softmax experiment used FP32, $R = 8S$, scale $1/\sqrt{64}$, and $S \in \{128, 255, 512, 768, 1024, 1536, 2048\}$. Attention used batch 1, eight heads, and head dimension 64. Inputs, masks, and allocations were outside the timed interval. Each case used 25 untimed warmups followed by 100 CUDA-event samples. The median was the primary latency statistic, with the 25th and 75th percentiles retained. CUDA synchronization ensured that device completion was included. The initial `torch.compile` call was excluded from steady-state latency and recorded as a separate compile warmup [23]–[26].

Every result row records its Git revision, implementation, shape, dtype, warmups, iterations, device, compute capability, PyTorch version, CUDA version, and timestamp. Raw samples are preserved rather than replaced by summaries.

The session is the experimental unit for claims beyond the captured run, so $n_{\text{session}} = 1$. The 100 observations per configuration estimate conditional within-session variation and are not 100 independent experimental replications. Reported dispersion is

$$\text{IQR} = Q_3 - Q_1, \quad \text{rIQR} = \text{IQR}/\text{median}. \quad (7)$$

Per-shape speedup is the baseline median divided by the candidate median; aggregate speedups use the geometric mean of the seven ratios. The conditional rank statistic

$$A_{12}^< = P(T_A < T_B) + \frac{1}{2}P(T_A = T_B) \quad (8)$$

is estimated from all observed pairs. It describes separation within this session; it is not accompanied by a

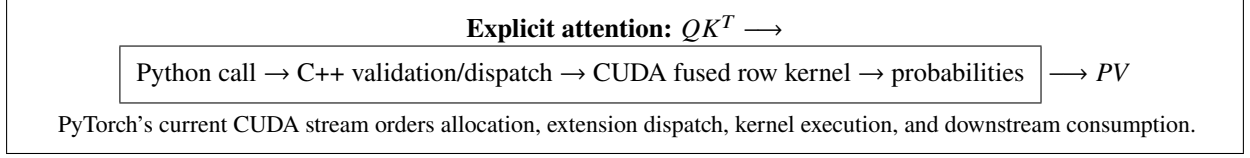


Figure 1. Software boundary of the fused operator. Only scale, causal masking, and stable softmax are fused; score and value matrix multiplications remain materialized in the explicit-attention paths.

Algorithm 1: Fused causal scaled softmax

1. Assign block r to row r ; set $q = r \bmod S$.
2. Each thread stages allowed αX_{rj} in output scratch, writes zero for $j > q$, and accumulates a local maximum.
3. Reduce maxima within warps; publish one value per warp; let the first warp reduce those partials to obtain m_r .
4. Each thread overwrites allowed scratch values with $\exp(\alpha X_{rj} - m_r)$ and accumulates a local sum.
5. Repeat the two-level warp reduction to obtain denominator z_r .
6. Each thread overwrites its allowed values with Y_{rj}/z_r .

Pseudocode for the final kernel. The output allocation is reused as scratch for scaled values and stable exponentials.

population-level p -value. Log-log slopes and the Amdahl calculation are diagnostic trends, not complexity proofs or fitted predictive models.

4. CUDA IMPLEMENTATION EVOLUTION

4.1. Row-Serial Baseline

The first complete design assigned one softmax row to one CUDA thread. That thread scanned allowed columns for the maximum, scanned again for the stable exponential sum, and normalized the row. The mapping was simple and provided a correct baseline, but exposed no parallelism inside a row.

4.2. Block-Parallel Shared Trees

The second design assigned one block to one row. Thread t visited columns $t + k \text{ blockDim.x}$, giving neighboring threads neighboring initial addresses while supporting arbitrary row lengths. Each thread accumulated register-local maximum and denominator partials. Full shared-memory trees then combined those partials, with block barriers protecting producer-consumer boundaries. A second strided pass normalized output columns in parallel.

4.3. Compact Warp Reductions

The final design used `__shfl_down_sync` for maximum and sum reductions within each warp. Each warp leader wrote one value to a compact shared array; after one block barrier, the first warp completed the cross-warp reduction.

For a 128-thread block, four warp partials require 16 bytes of dynamic shared memory. Scaling, the causal predicate, stable maximum subtraction, exponentiation, both reductions, and normalization remain fused in one kernel.

Historical algorithms were recovered from commits 8f07d762 (row serial), f9420de0 (shared tree), and a3736910 (warp reduction). The historical rebuild applied a recorded header-only include needed by CUDA 12.8; no algorithm, workload, or tolerance was changed.

5. EXPERIMENTAL SETUP

Measurements were collected in one Google Colab session. Table 1 summarizes the captured environment. The NVIDIA driver advertised CUDA 13.0 compatibility, while the installed toolkit and PyTorch build used CUDA 12.8. Current framework and attention benchmarks correspond to clean commit ca87722a00ebf585cd788c67949e7c0b32dca788.

6. RESULTS

6.1. Correctness

The CUDA test stage reported 70 passes. The structured softmax matrix contained 88 comparisons (11 lengths times 8 input families), all of which passed. The maximum absolute error was 3.5762787×10^{-7} , maximum relative error was 4.6973432×10^{-7} , and maximum row-sum error was 3.5762787×10^{-7} . Every masked future probability was exactly zero, and no unexpected NaN or infinity appeared. Table 2 summarizes the accepted evidence.

All seven complete-attention comparisons passed. Maximum absolute output error was 2.9802322×10^{-7} for custom explicit attention versus explicit PyTorch and 7.1525574×10^{-7} for SDPA versus the same reference.

6.2. Historical Kernel Evolution (RQ2)

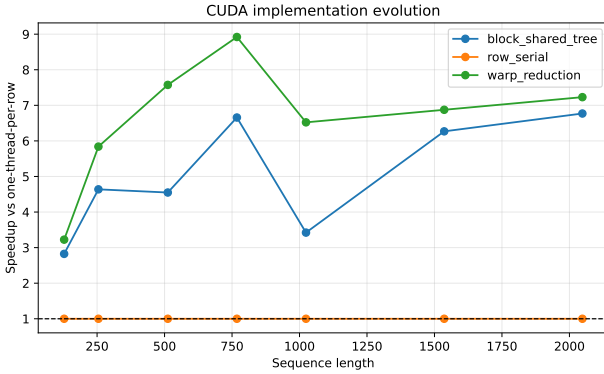
Table 3 and Figure 2 show the matched historical comparison. The warp kernel was 3.22–8.92 $\times$ faster than row serial and 1.07–1.91 $\times$ faster than the shared-tree design across all tested lengths. The larger row-serial gap combines work-decomposition and reduction improvements; the shared-tree comparison more closely isolates the communication change after block-per-row work was already present.

Table 1. Measured execution environment and controlled workload. CUDA-event observations are repeated measurements inside one session, not independent replications.

Category	Configuration	Recorded value
Hardware	GPU	NVIDIA Tesla T4; compute capability 7.5; 15.6 GB [27]
Host	Runtime	Google Colab; Linux 6.6.122; x86-64
Software	Toolchain	Python 3.13.15; PyTorch 2.11.0+cu128; NVCC 12.8.93; GCC 11.4.0
Software	Driver	580.82.07; advertised CUDA compatibility 13.0
Softmax geometry	Matrix and scale	$R = 8S$ rows by S columns; $\alpha = 1/\sqrt{64}$
Attention geometry	Dimensions	batch 1; 8 heads; head dimension 64
Workload	Sequence lengths	128, 255, 512, 768, 1024, 1536, 2048; FP32
Launch sweep	Threads per block	128, 256, 512
Timing	Protocol	25 warmups; 100 one-iteration CUDA-event observations
Correctness	Criteria	$\text{rtol} = 10^{-5}$; $\text{atol} = 10^{-6}$; seed 1234
Provenance	Measured revision	ca87722a00ebf585cd788c67949e7c0b32dca788

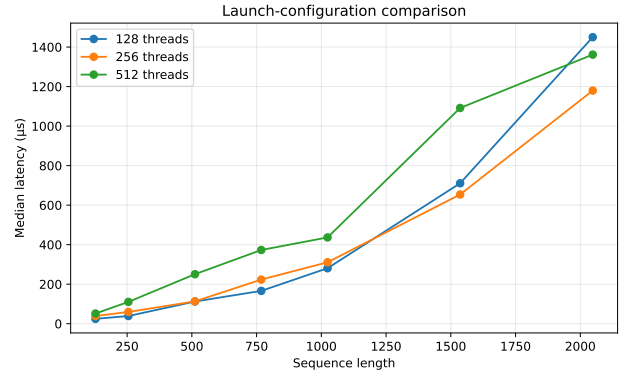
Table 2. Correctness evidence summary at fixed FP32 tolerances.

Check	Softmax	Attention
Cases passed	88/88	7/7
Maximum absolute error	3.5763×10^{-7}	2.9802×10^{-7}
Maximum row-sum error	3.5763×10^{-7}	—
Masked future values	all zero	causal outputs passed
Unexpected NaN/Inf	none	none

**Figure 2.** Historical kernel speedup from matched raw CUDA-event samples. Row serial (8f07d762) is the reference for the plotted ratios.

6.3. Launch Configuration (RQ3)

The 128-thread block had the lowest median at lengths 128, 255, 512, 768, and 1024. A 256-thread block won at 1536 and 2048; 512 threads won no tested shape. The predefined aggregate rule normalized latency by the fastest configuration at each length and selected the block size with the lowest median relative latency. It selected 128 threads with score 1.000, compared with 1.107 for 256 and 2.100 for 512. Figure 3 demonstrates that the per-shape winner is not universal.

**Figure 3.** Launch-configuration median latency on the Tesla T4. Only block size changes within this comparison.

Using a geometric-mean penalty against a per-shape oracle, fixed launches of 128, 256, and 512 threads score 1.042, 1.202, and 1.900, respectively. Thus, 128 threads are the best measured fixed policy, but a threshold policy using 256 threads for $S \geq 1536$ reproduces the oracle only for these seven T4 points; it is not asserted as a portable rule.

6.4. Framework Softmax Comparison (RQ4)

Table 5 and Figure 4 compare equivalent scale-mask-softmax work. The custom operator beat eager PyTorch at all seven lengths by 1.40–3.94 $\times$. It beat steady-state torch.compile at six lengths; at $S = 2048$, compiled PyTorch was approximately 2.1% faster. The geometric-mean custom-over-eager speedup was 2.73 $\times$. Fitted log-log latency slopes were 1.294 for custom, 1.622 for eager, and 0.761 for compiled PyTorch over these seven points; fixed overhead and limited range prevent interpreting them as asymptotic exponents.

The observed custom/eager rank separation was 0.9724 at $S = 128$ and complete at the reported precision for all other lengths. Relative IQR ranged from 1.75–21.72% for custom, 0.57–15.04% for eager, and 1.78–22.86% for the compiled path. At $S = 2048$, the compiled median was

Table 3. Historical-kernel ablation on the Tesla T4. Latencies are medians in microseconds; ratios greater than one favor the warp-reduction kernel.

S	Row serial	Shared tree	Warp reduction	Row/warp	Shared/warp
128	149.664	52.992	46.416	3.22×	1.14×
255	371.120	80.016	63.552	5.84×	1.26×
512	977.248	214.832	129.024	7.57×	1.67×
768	2101.248	315.632	235.552	8.92×	1.34×
1024	2032.896	594.016	311.728	6.52×	1.91×
1536	4595.920	733.184	668.560	6.87×	1.10×
2048	8827.104	1303.936	1220.928	7.23×	1.07×
Geometric mean	—	—	—	6.34×	1.32×

Table 4. Launch-configuration median latency in microseconds. Bold values are the lowest observed medians at each sequence length.

S	128 threads	256 threads	512 threads	Best launch
128	24.528	38.816	51.504	128
255	38.912	59.584	110.432	128
512	112.256	112.912	250.464	128
768	166.048	223.216	372.992	128
1024	280.656	310.768	436.576	128
1536	710.656	653.824	1091.760	256
2048	1449.984	1179.760	1361.744	256

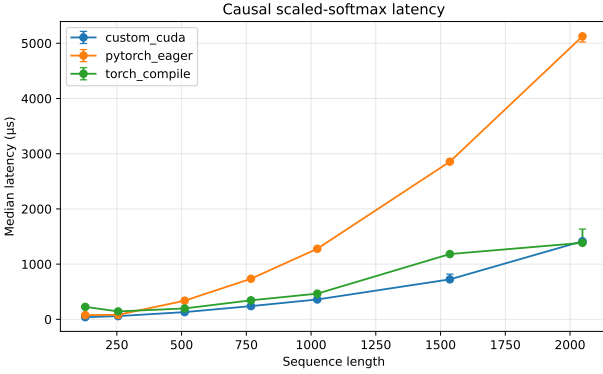


Figure 4. Median softmax latency for custom CUDA and framework baselines.

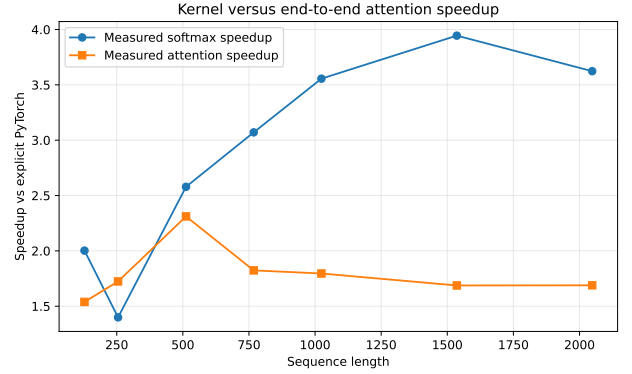


Figure 5. Isolated-softmax and explicit-attention speedup over their respective eager PyTorch baselines.

lower and its conditional $A_{12}^<$ against custom was 0.4385, so universal custom superiority is not supported. These effect sizes remain descriptive of one session.

6.5. Complete Attention (RQ5)

The explicit custom path computes QK^T , invokes the custom softmax, and then computes PV . It was 1.54–2.31× faster than explicit eager attention. Kernel speedup exceeded attention speedup at six of seven lengths, consistent with the surrounding matrix multiplications limiting the benefit of optimizing only softmax. Production SDPA remained 1.25–2.53× faster than the explicit custom path at every length. This is a mathematically comparable output baseline, but not an implementation-equivalent fusion boundary: SDPA can fuse a larger portion of attention.

Conditional $A_{12}^<$ for custom versus eager explicit attention ranged from 0.9931 to 1.0000. Conversely, the

probability of an observed SDPA timing below custom was 1.0000 at six lengths and 0.9998 at $S = 512$. These values describe near-complete separation inside the captured session and do not convert one session into a replicated population study.

6.5.1. Amdahl-Model Diagnostic

If fraction f of baseline attention time is accelerated by isolated factor s , the simple model predicts

$$S_{\text{total}} = \frac{1}{(1-f) + f/s}. \quad (9)$$

Table 7 compares Equation 9 with measurement. The model underpredicts the three shortest shapes, approximately matches $S = 768$, and overpredicts the three longest. Mean absolute error is 0.419×, root mean square error is 0.499×, and mean absolute percentage error is

Table 5. Isolated causal-softmax latency and conditional effect size. Each cell “median (IQR)” is in microseconds; $A_{12}^<$ is the probability that a sampled custom observation is below a sampled eager observation within the captured session.

S	Custom CUDA	PyTorch eager	torch.compile	Eager/custom	$A_{12}^<$
128	38.752 (8.416)	77.600 (11.672)	225.664 (30.040)	2.00×	0.9724
255	58.544 (2.496)	81.920 (10.392)	143.456 (32.792)	1.40×	1.0000
512	131.040 (12.888)	337.920 (4.016)	197.392 (37.000)	2.58×	1.0000
768	239.440 (12.984)	735.264 (4.984)	344.352 (22.896)	3.07×	1.0000
1024	359.568 (6.304)	1278.592 (7.256)	465.424 (29.304)	3.56×	1.0000
1536	723.952 (104.120)	2855.840 (20.072)	1182.416 (21.016)	3.94×	1.0000
2048	1414.480 (34.168)	5125.488 (116.352)	1385.760 (279.384)	3.62×	1.0000
Geometric mean	—	—	—	2.73×	—

Table 6. Complete-attention median latency and conditional effect size. The two explicit paths materialize score and probability matrices; SDPA may select a broader fused backend.

S	Custom explicit	Eager explicit	SDPA	Eager/custom	Custom/SDPA	$A_{12}^<$
128	129.008	198.544	84.000	1.54×	1.54×	0.9931
255	204.288	352.128	163.600	1.72×	1.25×	0.9983
512	323.584	747.888	245.808	2.31×	1.32×	1.0000
768	587.312	1071.088	384.864	1.82×	1.53×	1.0000
1024	1014.016	1820.672	543.744	1.80×	1.86×	1.0000
1536	2438.960	4115.824	1032.400	1.69×	2.36×	1.0000
2048	4198.768	7090.240	1660.576	1.69×	2.53×	1.0000
Geometric mean	—	—	—	1.78×	1.71×	—

22.7%. The residual pattern indicates nonadditive dispatch, allocation, materialization, or backend interactions; the model is diagnostic rather than fitted [28].

6.6. Profiling

Measured. At $S = 512$, PyTorch Profiler attributed 110.207 μs of CUDA time to the fused kernel. Named device totals were 505.565 μs for custom explicit attention, 1095.195 μs for explicit eager attention, and 408.222 μs for SDPA. Nsight Compute 2025.1.1 captured four target launches and reported 128 threads per block, 24 registers per thread, zero static shared memory, and 16 bytes of dynamic shared memory. Kernel durations were 106.976–107.584 μs , with 48.33–48.79% peak DRAM throughput and 56.72–57.00% peak SM throughput [29], [30].

Interpretation. The dynamic shared-memory footprint agrees with four warp partials for a 128-thread block. The throughput percentages do not prove a single memory or compute bottleneck. Profiler captures use different conditions from repeated benchmark medians and are not substituted for them.

Table 7. Simple Amdahl prediction versus measured explicit-attention speedup. The component fractions are measured values, and residual is measured minus predicted speedup.

S	Eager-softmax fraction f	Isolated speedup s	Prediction	Measured	Residual
128	0.3908	2.0025	1.2433	1.5390	+0.2957
255	0.2326	1.3993	1.0711	1.7237	+0.6526
512	0.4518	2.5788	1.3824	2.3113	+0.9289
768	0.6865	3.0708	1.8619	1.8237	-0.0382
1024	0.7023	3.5559	2.0193	1.7955	-0.2238
1536	0.6939	3.9448	2.0746	1.6875	-0.3870
2048	0.7229	3.6236	2.0982	1.6886	-0.4095

Table 8. PyTorch Profiler device time at $S = 512$. Path totals and the fused kernel row describe different scopes and should not be added.

Profiled scope	Device time (μs)
Custom explicit-attention total	505.565
Explicit eager-attention total	1095.195
PyTorch SDPA total	408.222
Fused causal-softmax kernel	110.207

Table 9. Evidence-bounded answers to the research questions.

Question	Principal evidence	Bounded answer
RQ1: correctness	88/88 softmax and 7/7 attention cases passed; maximum absolute error 3.5763×10^{-7} ; exact masked zeros	Correct for the tested FP32 forward domain within the declared tolerance
RQ2: ablation	6.34× geometric mean over row serial; 1.32× over shared tree	Work decomposition dominates; compact warp communication adds a smaller consistent gain
RQ3: launch	128 threads win 5/7 points; 256 wins 2/7; fixed-128 oracle penalty 4.2%	Launch is shape dependent; 128 is the best measured fixed T4 policy
RQ4: framework softmax	2.73× geometric mean over eager; compiled PyTorch wins slightly at $S = 2048$	Custom beats equivalent eager work, not every PyTorch implementation
RQ5: attention	1.78× geometric mean over explicit eager; custom is 1.71× slower than SDPA geometrically	Kernel gains translate partially; attention-wide fusion remains superior

7. DISCUSSION

7.1. Hypothesis Evaluation

Table 10 summarizes the evidence-bounded hypothesis outcome. The row-serial growth trend was only partially supported because latency trends do not alone prove the causal mechanism. Block-per-row work was supported at the tested long shapes. Warp reduction and shape-dependent launch choice were supported on the T4. The kernel-to-attention prediction was partially supported because the kernel speedup exceeded attention speedup at six, not all seven, matched lengths.

Table 10. Hypothesis status from the measured T4 experiment.

ID	Status	Evidence boundary
H1	Partially supported	Row-serial latency grew 58.98× versus 24.61× for block/shared tree.
H2	Supported	Block-per-row improved the tested long shapes by 3.42–6.77×.
H3	Supported	Warp was faster than shared tree at all seven lengths.
H4	Supported	128 and 256 threads each won at least one shape.
H5	Partially supported	Kernel speedup exceeded attention speedup for 6/7 lengths.

7.2. Microkernel Versus Application Performance

The historical comparison shows that work decomposition produced the largest gain: assigning a block rather than a thread to a row exposed intra-row parallelism. Warp communication then produced a smaller but consistently positive improvement over a block design that was already parallel.

Application improvement was smaller because the custom kernel changes only one stage. The diagnostic in Table 7 also shows that isolated component time does not compose exactly: launch overhead, backend selection, allocation, and materialization differ between isolated and integrated paths.

SDPA’s consistent advantage is a central result. The custom softmax improves an explicit attention composition, but a production backend can optimize and fuse across a broader boundary. A fast isolated component is therefore useful evidence, not proof of a superior end-to-end attention implementation.

8. THREATS TO VALIDITY AND PUBLICATION READINESS

8.1. Internal Validity

Implementation defects. Independent PyTorch compositions and edge cases reduce shared-bug risk but do not prove correctness. Independent reimplementation or formal kernel verification would strengthen confidence.

Historical reconstruction. The CUDA 12.8 header compatibility change is algorithmically inert but means historical build inputs are not byte-identical. Compiler changes may affect latency independently of the source algorithm.

Measurement state. One managed Colab session cannot fully control clocks, thermals, allocator state, caching, contention, or implementation order. Warmup reduces startup effects but cannot remove session drift. Profiling adds replay and instrumentation, so profiler totals are intentionally kept separate from benchmark medians.

8.2. External Validity

All quantitative results come from one Tesla T4. They do not establish transfer to Ampere, Ada, Hopper, Blackwell, AMD GPUs, or CPUs. PyTorch 2.11.0+cu128 and CUDA 12.8 define the measured software environment; compiled graphs and SDPA backend selection may change in later releases.

Only batch 1, eight heads, head dimension 64, FP32, causal masking, and seven sequence lengths were timed. The operator has no backward/autograd support, dropout, padding or arbitrary masks, variable valid lengths, or mixed precision. Conclusions do not extend to training or those workload dimensions.

8.3. Construct Validity

CUDA events measure device work ordered between events, not full request wall-clock latency, energy, peak memory, compilation cost, binary size, portability, or maintenance effort. Eager, compiled, custom, and SDPA paths implement comparable outputs but expose different fusion boundaries; each comparison answers a different engineering question.

The row-serial comparison bundles work ownership, memory access, reduction, and normalization changes. Shared tree versus warp reduction is the narrower communication ablation. Fixed-tolerance agreement is empirical rather than formal, and the absent Triton, CUB, mixed-precision, and backward baselines limit claims of library-level competitiveness.

8.4. Conclusion Validity

The central threat is pseudoreplication. The 100 timings are not 100 independent sessions, so $A_{12}^{\leq}$ and quartiles

describe only the captured conditional distributions. No population-level significance test or confidence interval is reported. Geometric means can conceal reversals such as the compiled result at $S = 2048$, so per-shape tables remain primary. Log-log slopes use seven points, and the Amdahl diagnostic uses component timings that need not compose additively.

A confirmatory design should run at least five, preferably ten or more, independently initialized sessions per GPU, randomize implementation order, and analyze paired session medians. A preregistered analysis could then report Wilcoxon signed-rank results, paired location and rank effect sizes, bootstrap confidence intervals, and Holm correction across primary comparisons. Those analyses are requirements for future replication, not claims about this run.

8.5. Publication-Readiness Assessment

The manuscript and artifact structure support a rigorous project report or preliminary systems study: explicit RQs, commit-linked evolution, independent correctness composition, raw timing samples, provenance, profiler exports, and direct validity limits. They do not guarantee journal acceptance or establish production superiority.

Before a journal submission, the highest-priority additions are independent randomized sessions; at least one Ampere-or-newer GPU; session-level inference; matched Triton, CUB, or specialized-library baselines; mixed precision and backward support if training relevance is claimed; matched profiler captures; and an immutable artifact release independently reproduced under artifact evaluation guidance [31].

9. CONCLUSION AND FUTURE WORK

On the measured Tesla T4 workload, changing from row-serial to block-per-row work delivered the largest kernel improvement, compact warp communication improved further on full shared-memory trees, and block-size preference varied with shape. The custom operator substantially outperformed equivalent eager softmax and accelerated explicit attention, but did not outperform production SDPA. The central systems result is that GPU mapping and communication choices matter locally, while end-to-end value depends on the fraction and fusion boundary of the larger application.

Future work should repeat the complete experiment across independent sessions and on a newer architecture; profile shared-tree and warp kernels under matched conditions; evaluate shape-aware launch dispatch; add FP16/BF16 accumulation analysis and fixed error criteria; add backward/autograd support; vary batch, head count, head dimension, and masking; and compare specialized library operators under explicitly equivalent work.

10. DECLARATIONS AND ARTIFACT AVAILABILITY

Author contribution (CRediT): Arwin Karir: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Visualization, Writing—Original Draft, and Writing—Review and Editing.

Conflict of interest: No conflict of interest is recorded for this independent educational research project.

Funding: No external funding is recorded for this project. The measured experiment used a Google Colab Tesla T4 runtime.

Data and code availability: Source, tests, notebook, raw CUDA-event samples, summaries, profiler exports, environment metadata, and generated figures are available in the public GitHub repository. The preserved evidence is in run 2026-08-23_tesla-t4_ca87722; its manifest records the full source revision ca87722a.

AI-assistance disclosure: OpenAI Codex was used to assist with manuscript restructuring, LaTeX formatting, copyediting, and repository build automation. It is not an author. The human author remains responsible for verifying the source code, citations, measurements, interpretations, and final submitted manuscript.

Ethics statement: This computer-systems study did not involve human participants, personal data, animals, or biological specimens.

Acknowledgments: The project relies on the open-source PyTorch ecosystem and NVIDIA CUDA development and profiling tools.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems* 30, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- [2] PyTorch Contributors. “Torch.nn.functional.scaled_dot_product_attention.” (2026), [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html (visited on 08/17/2026).
- [3] P. Blanchard, D. J. Higham, and N. J. Higham, “Accurate computation of the log-sum-exp and softmax functions,” *arXiv preprint arXiv:1909.03469*, 2019. doi: 10.48550/arXiv.1909.03469. [Online]. Available: <https://arxiv.org/abs/1909.03469>.
- [4] M. Milakov and N. Gimelshein, “Online normalizer calculation for softmax,” *arXiv preprint arXiv:1805.02867*, 2018. doi: 10.48550/arXiv.1805.02867. [Online]. Available: <https://arxiv.org/abs/1805.02867>.
- [5] NVIDIA Corporation. “Floating-point computation — CUDA programming guide.” (2026), [Online]. Available: <https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/mathematical-functions.html> (visited on 08/17/2026).

- [6] NVIDIA Corporation. “CUDA best practices guide.” (2026), [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html> (visited on 08/17/2026).
- [7] PyTorch Contributors. “Numerical accuracy.” (2026), [Online]. Available: https://docs.pytorch.org/docs/stable/notes/numerical_accuracy.html (visited on 08/17/2026).
- [8] PyTorch Contributors. “Reproducibility.” (2026), [Online]. Available: <https://docs.pytorch.org/docs/stable/notes/randomness.html> (visited on 08/17/2026).
- [9] NVIDIA Corporation. “CUDA programming guide.” (2026), [Online]. Available: <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html> (visited on 08/17/2026).
- [10] NVIDIA Corporation. “Warp shuffle functions — CUDA programming guide.” (2026), [Online]. Available: <https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html> (visited on 08/17/2026).
- [11] M. Harris, “Optimizing parallel reduction in CUDA,” NVIDIA Corporation, Tech. Rep., 2007. [Online]. Available: <https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>.
- [12] NVIDIA Corporation. “cub::BlockReduce: Block-wide reduction.” (2026), [Online]. Available: https://nvidia.github.io/cccl/cub/api/classcub_1_1BlockReduce.html (visited on 09/01/2026).
- [13] NVIDIA Corporation. “Occupancy — CUDA runtime api.” (2026), [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__OCCUPANCY.html (visited on 08/17/2026).
- [14] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems* 35, 2022. doi: 10.52202/068431-1189. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.
- [15] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- [16] M. N. Rabe and C. Staats, “Self-attention does not need $O(n^2)$ memory,” *arXiv preprint arXiv:2112.05682*, 2021. doi: 10.48550/arXiv.2112.05682.
- [17] S. Chetlur, C. Woolley, P. Vandermersch, *et al.*, “cuDNN: Efficient primitives for deep learning,” *arXiv preprint arXiv:1410.0759*, 2014. doi: 10.48550/arXiv.1410.0759.
- [18] P. Tillet, H. T. Kung, and D. Cox, “Triton: An intermediate language and compiler for tiled neural network computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19. doi: 10.1145/3315508.3329973.
- [19] J. Ansel *et al.*, “PyTorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024. doi: 10.1145/3620665.3640366.
- [20] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” In *Proceedings of ASPLOS*, 2009, pp. 265–276. doi: 10.1145/1508244.1508275.
- [21] C. Curtsinger and E. D. Berger, “STABILIZER: Statistically sound performance evaluation,” in *Proceedings of ASPLOS*, 2013, pp. 219–228. doi: 10.1145/2490301.2451141.
- [22] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. doi: 10.1145/1498765.1498785.
- [23] NVIDIA Corporation. “Event management — CUDA runtime api.” (2026), [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__EVENT.html (visited on 08/17/2026).
- [24] PyTorch Contributors. “Torch.cuda.event.” (2026), [Online]. Available: <https://docs.pytorch.org/docs/stable/generated/torch.cuda.Event.html> (visited on 08/17/2026).
- [25] PyTorch Contributors. “Torch.cuda.synchronize.” (2026), [Online]. Available: <https://docs.pytorch.org/docs/stable/generated/torch.cuda.synchronize.html> (visited on 08/17/2026).
- [26] PyTorch Contributors. “Torch.compile.” (2026), [Online]. Available: <https://docs.pytorch.org/docs/stable/generated/torch.compile.html> (visited on 08/17/2026).
- [27] NVIDIA Corporation. “NVIDIA T4 Tensor Core GPU product specifications.” (), [Online]. Available: <https://www.nvidia.com/en-us/data-center/tesla-t4/> (visited on 09/01/2026).
- [28] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, 1967, pp. 483–485. doi: 10.1145/1465482.1465560. [Online]. Available: <https://dl.acm.org/doi/10.1145/1465482.1465560>.
- [29] PyTorch Contributors. “PyTorch profiler.” (2026), [Online]. Available: https://docs.pytorch.org/tutorials/recipes/recipes/profiler_recipe.html (visited on 08/17/2026).
- [30] NVIDIA Corporation. “NVIDIA Nsight Compute profiling guide.” (2026), [Online]. Available: <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html> (visited on 08/17/2026).
- [31] Association for Computing Machinery. “Artifact review and badging.” version 1.1. (). [Online]. Available: <https://www.acm.org/publications/policies/artifact-review-and-badging-current> (visited on 09/01/2026).